MODULE 21 · DOMAIN 3 - DEPLOYMENT

AWS SAM and the CDK

Two higher-level ways to produce CloudFormation - and the four lines that give a Lambda function a canary deployment.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 21 of 34

admin@sthithapragna.com

SAM and the CDK are both **CloudFormation generators**. Every stack, change set and rollback rule from module 20 still applies underneath.

What SAM actually is

- A SAM template is a CloudFormation template with **Transform: AWS::Serverless-2016-10-31**.
- That macro expands SAM's short resource types into full CloudFormation resources.
- One **AWS::Serverless::Function** can expand into a function, a role, a log group and permissions.
- Ordinary CloudFormation resources can sit in the same template, unchanged.
- Without that Transform line, none of the SAM syntax is recognised.

> ON THE EXAM
>
> SAM is a dialect, not a different product. Anything CloudFormation can do, a SAM template can also do, by writing plain CloudFormation alongside.

The SAM resource types

TYPE	WHAT IT CREATES
AWS::Serverless::Function	A Lambda function, its role, log group and event source mappings
AWS::Serverless::Api	A REST API, from a template definition or OpenAPI
AWS::Serverless::HttpApi	An HTTP API
AWS::Serverless::SimpleTable	A DynamoDB table with a partition key only
AWS::Serverless::StateMachine	A Step Functions state machine
AWS::Serverless::LayerVersion	A Lambda layer
AWS::Serverless::Application	A nested application, often from the Serverless Application Repository

ON THE EXAM

SimpleTable is genuinely limited: partition key only, no sort key and no indexes. A real table needs AWS::DynamoDB::Table instead.

The Function resource

- **CodeUri**, **Handler** and **Runtime** are the essentials. Everything else has defaults.
- **Events** declares triggers, and SAM creates the plumbing each one needs.
- An **Api** event creates the API Gateway route, method and invoke permission automatically.
- An **SQS** event creates the event source mapping and adds the queue permissions to the role.
- **Policies** attaches IAM policies, including SAM's own policy templates.

> ON THE EXAM
>
> The Events section is where SAM saves the most work. Declaring a trigger generates the resource, the mapping and the permission, which is three things in CloudFormation.

Implicit APIs

- An **Api** event with no RestApiId creates an **implicit API** called ServerlessRestApi.
- Every function with an Api event joins that same implicit API automatically.
- So you get an API you never declared, with a logical ID you did not choose.
- Configure it through the **Globals** section, or declare an explicit AWS::Serverless::Api.
- Reference it as **!Ref ServerlessRestApi** if you need its ID in an output.

> ON THE EXAM
>
> The implicit API is convenient and invisible, which is exactly why it confuses people. Declaring an explicit API is clearer once a template grows.

Globals

- A **Globals** section sets properties inherited by every resource of a type.
- Typically Runtime, Timeout, MemorySize, Tracing and Environment for all functions.
- A property set on an individual resource **overrides** the global.
- Maps such as Environment variables are **merged**, not replaced. Scalars are replaced.
- It applies to Function, Api, HttpApi, SimpleTable and a few others.

> ON THE EXAM
>
> The merge behaviour catches people. A function that defines its own environment variables still inherits the global ones, which is usually what you wanted.

Policy templates

- Named, parameterised policies that expand into scoped IAM statements.
- **DynamoDBCrudPolicy**, **S3ReadPolicy**, **SQSSendMessagePolicy** and dozens more.
- Each takes a parameter, so the policy is scoped to one named resource.
- They are convenient rather than minimal. CrudPolicy grants more than a read-only function needs.
- Inline policy documents and managed policy ARNs still work in the same Policies list.

ON THE EXAM

Policy templates are the right default because they scope to a specific resource. Compare with AdministratorAccess, which is what unhurried developers reach for instead.

The SAM workflow

01

sam init

Scaffold a project from a template.

02

sam build

Install dependencies and stage artifacts in .aws-sam.

03

sam deploy

Package, upload to S3, and create a change set to execute.

04

sam logs / sam local

Tail logs, or invoke the function on your machine.

ON THE EXAM

sam deploy --guided runs once and writes samconfig.toml, so later deploys need no arguments. That file is what makes deployment repeatable.

Testing locally

- **sam local invoke** runs one function in a Docker container that mimics the Lambda runtime.
- **sam local start-api** runs an API Gateway emulator on localhost.
- **sam local start-lambda** exposes a local Lambda endpoint the SDK can call.
- **sam local generate-event** produces realistic sample events for dozens of sources.
- It emulates the runtime, **not the platform**. IAM, throttling and cold starts are not reproduced.

> ON THE EXAM
>
> That last line matters. Local testing catches logic errors quickly and will never catch a missing IAM permission, because locally there is no IAM.

sam sync and SAM Accelerate

- A full build and deploy cycle takes a minute or two, which is painful fifty times a day.
- **sam sync** pushes code changes directly to the deployed resources, bypassing CloudFormation.
- **sam sync --watch** monitors your files and syncs automatically on save.
- Code-only changes take seconds. Template changes still trigger a CloudFormation deployment.
- **For development only.** It deliberately bypasses change sets, so the stack can drift.

> ON THE EXAM
>
> This is the answer for 'our feedback loop is too slow' - and never the answer for production, because bypassing CloudFormation is the whole mechanism.

AutoPublishAlias

- Setting **AutoPublishAlias: live** on a function changes what every deployment does.
- SAM detects the code has changed and **publishes a new version**.
- It creates the alias if needed, and **points it at that new version**.
- It repoints **every event source** at the alias rather than at $LATEST.
- It is a prerequisite for DeploymentPreference. Without it, that property errors.

ON THE EXAM

This single line is what makes versioned, alias-based deployment the default rather than something you wire up by hand. Module 24 covers the underlying mechanics.

DeploymentPreference

- Adding **DeploymentPreference** makes SAM create a **CodeDeploy** deployment group automatically.
- **Canary10Percent5Minutes** shifts 10% of traffic, waits, then shifts the rest.
- **Linear10PercentEvery1Minute** shifts in equal increments. **AllAtOnce** does not shift at all.
- **Alarms** lists CloudWatch alarms. If one fires, CodeDeploy **rolls back immediately**.
- **Hooks** run PreTraffic and PostTraffic validation functions around the shift.

> READ THIS TWICE
>
> Four lines of YAML give you gradual traffic shifting, alarm-triggered automatic rollback and validation hooks. Nothing else on this exam buys as much for as little.

What SAM generates underneath

- **sam build** then **sam deploy** produces an ordinary CloudFormation stack.
- Inspect the expanded template in the console, or with **aws cloudformation get-template**.
- A twenty-line SAM template routinely becomes several hundred lines of CloudFormation.
- Stack events reference the **generated** logical IDs, not your SAM ones.
- So a failure names something like MyFunctionRole, which you never wrote.

ON THE EXAM

Knowing SAM expands into CloudFormation is what lets you debug it. The error messages come from CloudFormation and name resources SAM invented.

The AWS CDK

- Define infrastructure in TypeScript, Python, Java, C# or Go. It **synthesises** CloudFormation.
- So you get loops, conditionals, functions, types and an IDE that autocompletes.
- Reusable abstractions are ordinary classes, shared as ordinary packages.
- **CDK v2** ships as one package, **aws-cdk-lib**, plus the **constructs** library.
- The deployed artefact is still a CloudFormation template. Nothing bypasses it.

> ON THE EXAM
>
> The advantage is abstraction and testing. The cost is that a generated template is further from what you wrote, which makes some failures harder to trace.

The three construct levels

LEVEL	WHAT IT GIVES YOU
L1, Cfn prefix	A direct one-to-one mapping of a CloudFormation resource. No defaults
L2	A curated class with sensible defaults, helper methods and IAM grants
L3, patterns	A whole architecture in one construct. Several resources wired together

ON THE EXAM

L2 is where you normally work. grantRead on a bucket writes the IAM policy for you, which is the single biggest practical saving.

App, stack and construct

- An **App** contains one or more **Stacks**, and each stack becomes a CloudFormation stack.
- Stacks contain constructs, which contain other constructs, forming a tree.
- Every construct has a **construct ID**, and its path in the tree generates the logical ID.
- **Renaming or moving a construct changes its logical ID**, which CloudFormation reads as delete and recreate.
- Passing one construct to another creates the reference, and CDK adds the dependency.

> ON THE EXAM
>
> That fourth point causes real outages. A tidy-up refactor that renames a construct can silently replace a database, because the logical ID moved.

The CDK workflow

01

cdk bootstrap

Create the CDKToolkit stack in each account and Region. Once only.

02

cdk synth

Generate the CloudFormation template into cdk.out.

03

cdk diff

Compare the synthesised template against the deployed stack.

04

cdk deploy

Upload assets and deploy the stack.

ON THE EXAM

cdk diff is the CDK's change set: it shows what would change, including resource replacements, before you deploy anything.

Bootstrapping

- Creates a **CDKToolkit** stack holding an S3 bucket, an ECR repository and IAM roles.
- The bucket stores **assets**: function code, Docker images and large templates.
- Required **once per account and Region**. Deploying without it fails immediately.
- The bootstrap version can go stale, and a newer CDK will refuse to deploy until you re-bootstrap.
- Cross-account deployment needs the target account bootstrapped to trust the deploying account.

> ON THE EXAM
>
> 'This stack uses assets, so the toolkit stack must be deployed' is the error, and it means exactly one thing: run cdk bootstrap.

Escape hatches

- An L2 construct may not surface a property CloudFormation supports, especially a new one.
- Reach the underlying L1 with **node.defaultChild**, cast it, and set the property directly.
- **addPropertyOverride** and **addOverride** edit the synthesised template directly.
- **addDeletionOverride** removes a property the construct added.
- **Aspects** visit every construct in the tree, which is how you apply a tag or a check globally.

> ON THE EXAM
>
> Escape hatches are why an L2 construct is never a dead end. If the CDK is behind CloudFormation, you can still set the property.

Testing CDK code

- **Fine-grained assertions** check the synthesised template contains what you expect.
- Assert a bucket has encryption enabled, or a function has the right timeout.
- **Snapshot tests** compare the whole template against a stored copy, catching unintended changes.
- These run in your normal unit test framework, in seconds, with nothing deployed.
- They test the template, not the deployment. A valid template can still fail to deploy.

> ON THE EXAM
>
> This is the CDK's clearest advantage over SAM and raw CloudFormation: infrastructure that can be unit tested in an ordinary CI pipeline.

CDK Pipelines and migration

- **CDK Pipelines** defines a CodePipeline in CDK code that deploys the CDK app.
- It is **self-mutating**: adding a stage updates the pipeline on its next run.
- **cdk migrate** generates a CDK app from an existing stack or template.
- It pairs with CloudFormation's IaC generator from module 20.
- Generated code is a starting point. It uses L1 constructs and needs refactoring.

> ON THE EXAM
>
> Self-mutation is the notable idea: the pipeline is infrastructure too, so it deploys its own changes rather than needing manual edits.

SAM or the CDK?

AWS SAM

Serverless, declaratively

- YAML, close to CloudFormation
- Shortest path for Lambda and API Gateway
- Excellent local testing with sam local
- AutoPublishAlias and DeploymentPreference
- Limited outside serverless

AWS CDK

Anything, programmatically

- A real language, with types and loops
- Covers every AWS service
- Unit testable with assertions
- Reusable constructs as packages
- More to learn, and more to go wrong

THE TIEBREAKER

Purely serverless with a small team means SAM. Mixed infrastructure, shared abstractions or a team that wants tests means the CDK. They can be combined.

Which tool does the scenario want?

THE SCENARIO SAYS	THE INTENDED ANSWER
Gradual Lambda rollout with automatic rollback	SAM AutoPublishAlias and DeploymentPreference
Test a Lambda function on a developer machine	sam local invoke
Deployment feedback loop is too slow	sam sync, for development only
Unit test infrastructure definitions	CDK assertions
A property the L2 construct does not expose	A CDK escape hatch to L1
Existing resources, want them as code	IaC generator, or cdk migrate
Preview changes before applying them	cdk diff, or a CloudFormation change set

ON THE EXAM

Worth screenshotting. Most SAM and CDK questions reduce to one row, and most name a capability that only one tool has.

Two of these name a capability only one tool has. Find that capability before you weigh the tools.

A payments Lambda function is deployed by SAM. Deployments currently replace the code for all traffic at once, and a recent bad release caused 20 minutes of failed payments before anyone noticed.

The team wants new versions to take a small share of traffic first, and to revert automatically if the error rate alarm fires. They want the smallest possible change to the template.

What should be added to the template?

A. AutoPublishAlias and a DeploymentPreference of Canary10Percent5Minutes, with the error alarm listed

B. A second Lambda function and a Route 53 weighted record between them

C. Provisioned concurrency on the function, so new code warms before receiving traffic

D. An API Gateway canary release deployment on the stage

◆ PAUSE AND DECIDE

What should be added to the template?

A AutoPublishAlias plus DeploymentPreference

Four lines. SAM creates the CodeDeploy deployment group, shifts 10% of traffic for five minutes, and rolls back automatically if the alarm fires.

B Two functions and weighted DNS

Builds traffic shifting by hand, with two functions to keep in step and DNS TTLs that make rollback slow rather than immediate.

C Provisioned concurrency

Addresses cold starts, not release safety. Warm bad code fails just as reliably as cold bad code.

D API Gateway canary

Shifts traffic between API deployments, not between Lambda versions. It also does not roll back on a CloudWatch alarm.

THE SIGNAL

The requirement is traffic shifting at the Lambda layer with alarm rollback. Module 11 flagged the API Gateway canary as a different layer, and this is where that distinction pays.

> A developer is iterating on a Lambda function's business logic. Each change requires sam build and sam deploy, taking about 90 seconds before they can test against real DynamoDB and SQS resources in a development account.
>
> They make dozens of changes an hour. Local emulation is not sufficient because the logic depends on real service behaviour.

What should the developer use?

A sam local start-api, with the SDK pointed at the development account

B Reduce the deploy time by shrinking the deployment package

C sam sync --watch, accepting that it bypasses CloudFormation during development

D Deploy directly with the Lambda UpdateFunctionCode API from a script

◆ PAUSE AND DECIDE

What should the developer use?

A sam local start-api

The scenario rules this out: local emulation is explicitly insufficient because the logic depends on real service behaviour.

B Smaller package

Most of the 90 seconds is CloudFormation change set creation and execution, not upload. A smaller package barely moves it.

C sam sync --watch

Purpose-built for this: pushes code changes in seconds, watches files automatically, and the CloudFormation bypass is acceptable in a development account.

D UpdateFunctionCode script

Hand-builds what sam sync does, without dependency handling or multi-function support, and with the same drift downside.

THE SIGNAL

Note what makes C acceptable: a development account. The same answer would be wrong for production, because bypassing CloudFormation is the mechanism.

A team uses the CDK in TypeScript. They need to set a newly released property on an S3 bucket that the L2 Bucket construct does not yet expose. The property is fully supported by CloudFormation.

They do not want to abandon the L2 construct, which they use for its grant methods and defaults elsewhere in the stack.

How should they set the property?

A Replace the L2 Bucket with a CfnBucket and set every property manually

B Access the underlying CfnBucket through node.defaultChild and use addPropertyOverride

C Write the bucket in a separate raw CloudFormation template and deploy it alongside

D Wait for the CDK library to add support in a future release

◆ PAUSE AND DECIDE

How should they set the property?

A Drop to CfnBucket entirely

Works, and discards the defaults and grant methods the scenario says they want. An escape hatch keeps both.

B defaultChild plus addPropertyOverride

Exactly what escape hatches are for. The L2 construct is kept, and the property is written straight into the synthesised template.

C Separate raw template

Splits one bucket out of the stack that references it, creating a cross-stack dependency for a single property.

D Wait

Not a solution. Escape hatches exist so that CDK library lag never blocks you.

THE SIGNAL

The clause 'they do not want to abandon the L2 construct' is what separates B from A. Both set the property; only one keeps the grant methods.

> A platform team must define infrastructure spanning Lambda functions, an ECS service, a VPC, RDS and several S3 buckets. They want reusable components shared across four product teams, and they want infrastructure covered by unit tests in the existing CI pipeline.
>
> The team is fluent in TypeScript and already writes tests for application code.

Which tool should they adopt?

A AWS SAM, using nested applications for the reusable components

B Raw CloudFormation YAML with nested stacks

C Elastic Beanstalk, to avoid infrastructure definitions entirely

D The AWS CDK in TypeScript, with shared constructs as an internal package

◆ PAUSE AND DECIDE

Which tool should they adopt?

A SAM with nested applications

SAM is built for serverless. A VPC, RDS and an ECS service mean writing plain CloudFormation for most of it, losing SAM's advantage.

B Raw CloudFormation

Covers everything and offers no unit testing and no real abstraction. Nested stacks share templates, not typed components.

C Elastic Beanstalk

Deploys an application onto managed EC2. It does not define a VPC, RDS or S3 buckets, which is the actual requirement.

D CDK in TypeScript

Covers every service, shared constructs are an ordinary package four teams can depend on, and assertions give unit tests in the existing pipeline.

THE SIGNAL

Three clauses select the CDK: mixed non-serverless infrastructure, reusable typed components, and unit tests. No other option offers all three.

What to carry forward

- **Both generate CloudFormation.** Every rule from module 20 still applies underneath.
- **AutoPublishAlias plus DeploymentPreference** gives canary shifting and alarm rollback in four lines.
- **sam local** tests logic. **sam sync** speeds the loop, for development only.
- CDK constructs come in three levels, and an **escape hatch** reaches L1 when L2 lags.
- Renaming a CDK construct changes its logical ID, which CloudFormation reads as replacement.

> ON THE EXAM
>
> Next: CodePipeline, CodeBuild and CodeArtifact – the pipeline that runs these tools rather than a developer running them by hand.